

GUÍA DE DESARROLLO

Arquitectura Monolítica 3 Capas + API REST

1. Objetivo de la guía

Esta guía permite que cada aprendiz:

- Organice sus requerimientos por módulos
- Defina endpoints claros (GET, POST, PUT, DELETE)
- Estructure controladores con lógica y SQL
- Use variables de entorno correctamente
- Implemente respuestas estándar del backend

Resultado esperado: **un desarrollo ordenado, entendible y mantenible**

2. Arquitectura del sistema

Estructura general

Frontend (Interfaz)



Backend (API REST)



Base de Datos

Capas

1. Frontend

- Consume la API
- Muestra datos al usuario

2. Backend (API REST)

- Recibe solicitudes (HTTP)
- Procesa lógica
- Ejecuta SQL
- Devuelve respuestas

3. Base de Datos

- Almacena información

- Responde consultas SQL

3. Organización por módulos

Antes de programar, cada aprendiz debe organizar su sistema.

Ejemplo

Módulo: Usuarios

- Registrar usuario
- Login

Módulo: Mascotas

- Registrar mascota
- Listar mascotas

Módulo: Citas

- Agendar cita
- Cancelar cita

4. De requerimientos a endpoints

Ejemplo completo

Requerimiento:

Registrar usuario

Endpoint definido

Método	Ruta	Descripción
POST	<i>/api/usuarios</i>	Crear usuario

Flujo

1. Frontend envía datos
2. Ruta recibe solicitud

3. Controlador procesa
4. Ejecuta SQL
5. Devuelve respuesta

5. Estructura del backend

```
/backend
├── config/
├── controllers/
├── routes/
├── db/
├── utils/
└── app.js
```

6. Variables de entorno (MUY IMPORTANTE)

Nunca se debe quemar información sensible en el código.

Archivo .env

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=123456
DB_NAME=mi_base
PORT=3000
```

Uso en código

```
import dotenv from "dotenv";
dotenv.config();

const dbConfig = {
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
};
```

7. Conexión a base de datos

```
import mysql from "mysql2";

const connection = mysql.createConnection(dbConfig);

export default connection;
```

8. Controladores (lógica + SQL)

Aquí ocurre el trabajo real

Ejemplo: Crear usuario

```
export const crearUsuario = (req, res) => {  
  const { nombre, correo, password } = req.body;  
  
  const sql = "INSERT INTO usuarios (nombre, correo, password) VALUES (?, ?, ?)";  
  
  connection.query(sql, [nombre, correo, password], (err, result) => {  
    if (err) {  
      return res.status(500).json({  
        status: "error",  
        message: "Error al crear usuario",  
        data: null  
      });  
    }  
  
    res.status(201).json({  
      status: "success",  
      message: "Usuario creado correctamente",  
      data: result  
    });  
  });  
};
```

9. Rutas (enlace con endpoints)

```
import express from "express";  
import { crearUsuario } from "../controllers/usuarios.controller.js";  
  
const router = express.Router();  
  
router.post("/usuarios", crearUsuario);  
  
export default router;
```

10. Respuesta unificada del backend

TODOS los endpoints deben responder igual

Estructura estándar

```
{  
  "status": "success | error",  
  "message": "Descripción",  
  "data": {}  
}
```

Ejemplo éxito

```
{  
  "status": "success",  
  "message": "Usuario creado",  
  "data": {  
    "id": 1  
  }  
}
```

Ejemplo error

```
{  
  "status": "error",  
  "message": "Correo ya existe",  
  "data": null  
}
```

11. Diseño de endpoints por aprendiz

Cada aprendiz debe completar esta tabla

Plantilla

Módulo	Requerimiento	Método	Endpoint	Controlador	Descripción
Usuarios	Registrar	POST	/api/usuarios	crearUsuario	Crea usuario

Ejemplo completo

Módulo	Requerimiento	Método	Endpoint	Controlador
Citas	Agendar	POST	/api/citas	crearCita

12. Flujo completo (OBLIGATORIO entender)

Frontend → Ruta → Controlador → SQL → Respuesta JSON

13. Buenas prácticas

Código limpio

- Nombres claros
- Funciones pequeñas
- Evitar duplicación

Seguridad

- Validar datos
- No exponer contraseñas
- Usar variables de entorno

Organización

- Separar rutas y controladores
- Agrupar por módulos

Respuestas consistentes

Siempre usar:

- status
- message
- data

14. Errores comunes

- ✗ SQL dentro de rutas
- ✗ Variables quemadas
- ✗ Respuestas diferentes en cada endpoint
- ✗ No validar datos
- ✗ Mezclar módulos

15. Actividad práctica

Objetivo

Cada aprendiz debe construir su módulo completo

Debe entregar:

1. Módulo definido
2. Tabla de endpoints
3. Controladores funcionando
4. Rutas conectadas
5. Pruebas con Cliente ApiRest (thunder client (VC), postman , insomnia)

16. Cierre pedagógico

No se trata solo de que funcione...
Se trata de que esté bien organizado

Un buen backend:

- Es claro
- Es consistente
- Es escalable

Qué debe recibir cada endpoint, qué debe validar y qué debe responder.

Esto les evita a los aprendices “programar a ciegas”.

ENFOQUE CLAVE

Antes de escribir código, cada aprendiz debe poder responder:

1. ¿Qué recibe mi endpoint?
2. ¿Qué valida?

3. ¿Qué hace en base de datos?
4. ¿Qué responde (éxito y error)?

ESTRUCTURA LÓGICA DE CADA ENDPOINT

Todos deben seguir este patrón:

1. Recibir datos
2. Validar datos
3. Ejecutar lógica (SQL)
4. Manejar errores
5. Responder (estructura estándar)

EJEMPLOS POR MÓDULO (LO MÁS IMPORTANTE)

MÓDULO: AUTENTICACIÓN

Endpoint: Registrar usuario

Entrada (request)

```
{  
  "nombre": "Juan",  
  "correo": "juan@email.com",  
  "password": "123456"  
}
```

Lógica esperada

1. Validar que todos los campos existan
2. Validar formato de correo
3. Verificar que el correo no exista
4. Insertar en base de datos

Respuesta exitosa

```
{  
  "status": "success",  
  "message": "Usuario registrado correctamente",  
  "data": {  
    "id": 10,  
    "nombre": "Juan",  
    "correo": "juan@email.com"  
  }  
}
```

✗ Respuesta error (correo duplicado)

```
{  
  "status": "error",  
  "message": "El correo ya está registrado",  
  "data": null  
}
```

✗ Error validación

```
{  
  "status": "error",  
  "message": "El correo es obligatorio",  
  "data": null  
}
```

Endpoint: Login

Entrada

```
{  
  "correo": "juan@email.com",  
  "password": "123456"  
}
```

Lógica

1. Validar campos
2. Buscar usuario
3. Comparar contraseña
4. Retornar datos (sin password)

Respuesta

```
{  
  "status": "success",  
  "message": "Login exitoso",  
  "data": {  
    "id": 10,  
    "nombre": "Juan"  
  }  
}
```

✗ Error

```
{  
  "status": "error",  
  "message": "Credenciales incorrectas",  
  "data": null  
}
```

MÓDULO: MASCOTAS

Endpoint: Registrar mascota

Entrada

```
{  
  "nombre": "Firulais",  
  "tipo": "Perro",  
  "edad": 3,  
  "usuario_id": 10  
}
```

Lógica

1. Validar campos
2. Validar que el usuario exista
3. Insertar mascota
4. Asociar al usuario

Respuesta

```
{  
  "status": "success",  
  "message": "Mascota registrada",  
  "data": {  
    "id": 5,  
    "nombre": "Firulais"  
  }  
}
```

✖ Error

```
{  
  "status": "error",  
  "message": "El usuario no existe",  
  "data": null  
}
```

Endpoint: Listar mascotas

Entrada

No body (solo usuario autenticado o id)

Lógica

1. Recibir usuario_id
2. Consultar mascotas del usuario
3. Retornar lista

Respuesta

```
{
  "status": "success",
  "message": "Lista de mascotas",
  "data": [
    {
      "id": 5,
      "nombre": "Firulais",
      "tipo": "Perro"
    },
    {
      "id": 6,
      "nombre": "Michi",
      "tipo": "Gato"
    }
  ]
}
```

MÓDULO: CITAS

Endpoint: Agendar cita

Entrada

```
{
  "mascota_id": 5,
  "fecha": "2026-05-10",
  "hora": "10:00"
}
```

Lógica

1. Validar campos
2. Validar que la mascota exista
3. Validar que no haya otra cita en ese horario
4. Insertar cita

Respuesta

```
{
  "status": "success",
  "message": "Cita agendada correctamente",
  "data": {
    "id": 20,
    "fecha": "2026-05-10",
    "hora": "10:00"
  }
}
```

✗ Error (horario ocupado)

```
{
  "status": "error",
  "message": "El horario ya está ocupado",
  "data": null
}
```

Endpoint: Cancelar cita

Entrada

```
{
  "cita_id": 20
}
```

Lógica

1. Validar que la cita exista
2. Cambiar estado a cancelada
3. Confirmar operación

Respuesta

```
{
  "status": "success",
  "message": "Cita cancelada",
  "data": null
}
```

PATRÓN MENTAL QUE DEBEN APRENDER

Cada endpoint SIEMPRE debe responder estas preguntas:

Pregunta	Ejemplo
¿Qué entra?	JSON del usuario
¿Qué valido?	Campos, reglas
¿Qué proceso?	SQL
¿Qué devuelvo?	JSON estándar

REGLAS DE ORO PARA LOS APRENDICES

1. Nunca responder sin estructura

Siempre:

```
{  
  "status": "",  
  "message": "",  
  "data": {}  
}
```

2. No exponer datos sensibles

- ✗ password
- ✓ solo datos necesarios

3. Mensajes claros

- ✗ "Error 500"
- ✓ "El correo ya está registrado"

4. Pensar antes de programar

Si no puedes escribir la respuesta...
No estás listo para codificar

PLANTILLA PARA QUE EL APRENDIZ LLENE

Módulo: _____

Endpoint: _____

Método:
Ruta:

Entrada
{ }

Lógica

- 1.
- 2.
- 3.

Respuesta exitosa
{ }

✗ Respuesta error
{ }

CIERRE (MUY IMPORTANTE)

Un buen desarrollador no empieza escribiendo código
Empieza definiendo **cómo debe comportarse el sistema**